
Université de nouakchott

Faculté des Sciences et techniques

Module : Recherche Opérationnelle (Licence)

Responsable : Dr. EL BENANY Med Mahmoud

Année : 2025–2026 **Réalisé par :**

Meimoune Hourme (C28190)

Zienebou Mamoune (C23341)

Aminetou Abdlhay (C28292)

Rapport de Projets

Recherche Opérationnelle

Sujets traités :

1. Méthode du Simplexe (*Catégorie 1 — Fondamentaux*)
2. Problème du Voyageur (TSP) (*Catégorie 2 — Logistique*)
3. Routage des camions d'eau — Nouakchott (*Catégorie 6 — RIM*)

Outils utilisés : Python **OR-Tools** (Google) · $\text{\LaTeX}$

Table des matières

1	Introduction générale	2
2	Projet 1 — Méthode du Simplexe	3
2.1	Description du problème	3
2.2	Formulation mathématique	3
2.2.1	Exemple numérique	3
2.3	Résolution pas à pas	3
2.4	Interprétation économique	4
2.5	Code Python — OR-Tools	4
3	Projet 2 — Problème du Voyageur de Commerce (TSP)	6
3.1	Description du problème	6
3.2	Formulation mathématique	6
3.3	Exemple numérique — 5 villes	6
3.4	Code Python — OR-Tools (Routing)	7
4	Projet 3 — Routage des Camions d'Eau à Nouakchott	9
4.1	Contexte et problématique	9
4.2	Données du problème	9
4.3	Solution optimale	10
4.4	Code Python — OR-Tools (CVRP)	10
4.5	Recommandations pour Nouakchott	12
5	Conclusion générale	13

1. Introduction générale

La **Recherche Opérationnelle (RO)** est une discipline scientifique qui applique des méthodes mathématiques et algorithmiques pour aider à prendre les *meilleures décisions* face à des problèmes complexes.

Ce rapport présente trois projets choisis dans trois catégories différentes, conformément aux consignes du module. Chaque projet comprend :

- Une description et une formulation mathématique du problème,
- Un exemple numérique résolu pas à pas,
- Une implémentation complète en Python avec la bibliothèque **OR-Tools** de Google.

Outil principal : Google OR-Tools

OR-Tools est une bibliothèque open-source développée par Google, spécialisée dans la résolution de problèmes d'optimisation combinatoire : programmation linéaire, routage de véhicules, ordonnancement, etc.

Installation : `pip install ortools`

2. Projet 1 — Méthode du Simplexe

Catégorie 1 : Sujets classiques et fondamentaux

2.1. Description du problème

Définition

La **méthode du Simplexe** est un algorithme itératif permettant de résoudre des problèmes de **Programmation Linéaire (PL)**. Elle explore les sommets du domaine admissible (polytope) pour trouver la solution optimale.

2.2. Formulation mathématique

Un problème de programmation linéaire se présente sous la forme standard :

$$\max \quad z = \mathbf{c}^\top \mathbf{x} \quad (1)$$

$$\text{sous } A\mathbf{x} \leq \mathbf{b}, \quad \mathbf{x} \geq \mathbf{0} \quad (2)$$

2.2.1. Exemple numérique

$$\begin{aligned} \max \quad & z = 3x + 4y \\ \text{s.c.} \quad & \begin{cases} x + 2y \leq 8 & (\text{ressource A}) \\ 3x + y \leq 9 & (\text{ressource B}) \\ x, y \geq 0 \end{cases} \end{aligned}$$

2.3. Résolution pas à pas

Étape 1 — Variables d'écart : On introduit $s_1, s_2 \geq 0$:

$$x + 2y + s_1 = 8, \quad 3x + y + s_2 = 9$$

Étape 2 — Tableau initial :

Base	x	y	s_1	s_2	b
s_1	1	2	1	0	8
s_2	3	1	0	1	9
$-z$	-3	-4	0	0	0

TABLE 1 – Tableau initial du Simplexe

Variable entrante : y (coefficient -4 , le plus négatif).

Ratios : $8/2 = 4$ et $9/1 = 9 \Rightarrow$ pivot sur la ligne 1.

Étape 3 — Après premier pivot :

Base	x	y	s_1	s_2	b
y	0.5	1	0.5	0	4
s_2	2.5	0	-0.5	1	5
$-z$	-1	0	2	0	16

TABLE 2 – Tableau après le premier pivot

Variable entrante : x (coefficient -1). Ratio min : $5/2.5 = 2 \Rightarrow$ pivot ligne 2.

Étape 4 — Tableau final (solution optimale) :

Base	x	y	s_1	s_2	b
y	0	1	0.6	-0.2	3
x	1	0	-0.2	0.4	2
$-z$	0	0	1.8	0.4	18

TABLE 3 – Tableau final — tous les coefficients de z sont ≥ 0

Résultat optimal

$$x^* = 2, \quad y^* = 3, \quad z_{\max} = 18$$

Toutes les ressources sont consommées entièrement ($s_1 = s_2 = 0$).

Prix-ombres : ressource A = 1.8, ressource B = 0.4.

2.4. Interprétation économique

- Produire **2 unités de X** et **3 unités de Y** donne un profit maximum de **18 MRU**.
- Le prix-ombre de la ressource A est 1.8 : une unité supplémentaire de A augmente le profit de 1.8 MRU.
- Le prix-ombre de la ressource B est 0.4 : une unité supplémentaire de B augmente le profit de 0.4 MRU.

2.5. Code Python — OR-Tools

```

1 from ortools.linear_solver import pywraplp
2
3 # Créer le solveur (GLOP = solveur Simplexe de Google)
4 solver = pywraplp.Solver.CreateSolver('GLOP')
5
6 # Variables de decision (>= 0 par défaut)
7 x = solver.NumVar(0, solver.infinity(), 'x')
8 y = solver.NumVar(0, solver.infinity(), 'y')
9
10 # Contraintes
11 solver.Add(x + 2*y <= 8)      # Ressource A
12 solver.Add(3*x + y <= 9)     # Ressource B
13
14 # Objectif : maximiser z = 3x + 4y
15 solver.Maximize(3*x + 4*y)

```

```
16
17 # Resolution
18 status = solver.Solve()
19
20 # Affichage des resultats
21 if status == pywraplp.Solver.OPTIMAL:
22     print("==== Solution Optimale ====")
23     print(f"x = {x.solution_value()}")
24     print(f"y = {y.solution_value()}")
25     print(f"Profit max z = {solver.Objective().Value()}")
26     print(f"Valeur duale contrainte 1 = {solver.constraint(0).dual_value()}")
27     print(f"Valeur duale contrainte 2 = {solver.constraint(1).dual_value()}")
28 else:
29     print("Pas de solution optimale trouvee.")
```

Listing 1 – Simplexe avec OR-Tools (GLOP Solver)

Résultat attendu :

```
==== Solution Optimale ====
x = 2.0
y = 3.0
Profit max z = 18.0
Valeur duale contrainte 1 = 1.8
Valeur duale contrainte 2 = 0.4
```

3. Projet 2 — Problème du Voyageur de Commerce (TSP)

Catégorie 2 : Logistique et supply chain

3.1. Description du problème

Définition

Le **Travelling Salesman Problem (TSP)** consiste à trouver le cycle hamiltonien de coût minimum, c'est-à-dire la tournée la plus courte visitant toutes les villes **exactement une fois** et revenant à la ville de départ.

C'est un problème **NP-difficile** : le nombre de tournées possibles pour n villes est $(n-1)!/2$.

3.2. Formulation mathématique

Soit n villes. On définit $x_{ij} \in \{0, 1\}$ tel que $x_{ij} = 1$ si le voyageur va de i à j .

$$\min \sum_{i=1}^n \sum_{j \neq i} d_{ij} x_{ij} \quad (3)$$

Sous les contraintes :

$$\sum_{j \neq i} x_{ij} = 1 \quad \forall i \quad (\text{quitter chaque ville une fois}) \quad (4)$$

$$\sum_{i \neq j} x_{ij} = 1 \quad \forall j \quad (\text{entrer dans chaque ville une fois}) \quad (5)$$

$$x_{ij} \in \{0, 1\} \quad (\text{variables binaires}) \quad (6)$$

Plus les contraintes **d'élimination des sous-tournées (SECs)**.

3.3. Exemple numérique — 5 villes

Ville	A	B	C	D	E
A	—	10	15	20	25
B	10	—	35	25	30
C	15	35	—	30	20
D	20	25	30	—	15
E	25	30	20	15	—

TABLE 4 – Matrice des distances symétriques (km)

Résultat optimal

Tournée optimale : $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow A$

Distance totale = $10 + 25 + 15 + 20 + 15 = \mathbf{85}$ km

3.4. Code Python — OR-Tools (Routing)

```

1 from ortools.constraint_solver import routing_enums_pb2
2 from ortools.constraint_solver import pywraprouting
3
4 # Matrice des distances (5 villes : A=0, B=1, C=2, D=3, E=4)
5 distance_matrix = [
6     [0, 10, 15, 20, 25], # A
7     [10, 0, 35, 25, 30], # B
8     [15, 35, 0, 30, 20], # C
9     [20, 25, 30, 0, 15], # D
10    [25, 30, 20, 15, 0], # E
11 ]
12
13 # Parametres du modele
14 num_villes = 5
15 depot = 0 # Ville de depart : A
16
17 # Creer le gestionnaire d'index
18 manager = pywraprouting.RoutingIndexManager(num_villes, 1, depot)
19 routing = pywraprouting.RoutingModel(manager)
20
21 # Fonction de cout (distance entre deux villes)
22 def distance_callback(from_index, to_index):
23     from_node = manager.IndexToNode(from_index)
24     to_node = manager.IndexToNode(to_index)
25     return distance_matrix[from_node][to_node]
26
27 transit_callback_index = routing.RegisterTransitCallback(
28     distance_callback)
29 routing.SetArcCostEvaluatorOfAllVehicles(transit_callback_index)
30
31 # Parametres de recherche (heuristique initiale + optimisation
32 # locale)
33 search_params = pywraprouting.DefaultRoutingSearchParameters()
34 search_params.first_solution_strategy = (
35     routing_enums_pb2.FirstSolutionStrategy.PATH_CHEAPEST_ARC
36 )
37 search_params.local_search_metaheuristic = (
38     routing_enums_pb2.LocalSearchMetaheuristic.GUIDED_LOCAL_SEARCH
39 )
40 search_params.time_limit.seconds = 5
41
42 # Resolution
43 solution = routing.SolveWithParameters(search_params)
44
45 # Affichage de la tournée
46 if solution:
47     villes = ['A', 'B', 'C', 'D', 'E']
48     print("==== Solution TSP =====")
49     index = routing.Start(0)

```



```
48     tournée = []
49     distance_totale = 0
50     while not routing.IsEnd(index):
51         tournée.append(villes[manager.IndexToNode(index)])
52         previous_index = index
53         index = solution.Value(routing.NextVar(index))
54         distance_totale += routing.GetArcCostForVehicle(
55             previous_index, index, 0)
56     tournée.append(villes[manager.IndexToNode(index)])
57     print(" -> ".join(tournée))
58     print(f"Distance totale : {distance_totale} km")
```

Listing 2 – TSP avec OR-Tools Routing Library

Résultat attendu :

```
===== Solution TSP =====
A -> B -> D -> E -> C -> A
Distance totale : 85 km
```

4. Projet 3 — Routage des Camions d'Eau à Nouakchott

Catégorie 6 : Problèmes réels (RIM) — Obligatoire

4.1. Contexte et problématique

Dans plusieurs quartiers périphériques de **Nouakchott**, la distribution d'eau potable dépend de camions-citernes partant d'un dépôt central. Ce problème est modélisé comme un **VRP avec contraintes de capacité (CVRP)**.

Vehicle Routing Problem (VRP)

Le VRP consiste à planifier les tournées d'une flotte de véhicules depuis un dépôt pour desservir un ensemble de clients, en **minimisant la distance totale** tout en respectant la **capacité de chaque véhicule**.

4.2. Données du problème

- **1 dépôt central** (nœud 0) à Nouakchott
- **6 points de distribution** : P1 à P6 (quartiers périphériques)
- **Capacité de chaque camion** : 1000 litres
- **Nombre de camions disponibles** : 2

Point	Demande (litres)	Quartier
P1	200	Arafat
P2	300	Sebkha
P3	250	Dar Naim
P4	150	Teyarett
P5	400	Toujounine
P6	350	Riyad
Total	1650	—

TABLE 5 – Demandes en eau des quartiers périphériques de Nouakchott

	D	P1	P2	P3	P4	P5	P6
D	0	3	5	8	6	10	12
P1	3	0	4	7	5	9	11
P2	5	4	0	3	6	7	8
P3	8	7	3	0	4	5	6
P4	6	5	6	4	0	8	9
P5	10	9	7	5	8	0	4
P6	12	11	8	6	9	4	0

TABLE 6 – Matrice des distances (km) entre le dépôt et les points de distribution

4.3. Solution optimale

Camion	Tournée	Charge (L)	Distance (km)
Camion 1	$D \rightarrow P1 \rightarrow P2 \rightarrow P4 \rightarrow D$	650 / 1000	19
Camion 2	$D \rightarrow P3 \rightarrow P5 \rightarrow P6 \rightarrow D$	1000 / 1000	29
Total	—	1650 L	48 km

TABLE 7 – Plan de tournées optimal pour la distribution d'eau

Résultat optimal

Deux camions suffisent pour desservir les 6 quartiers.
 Distance totale minimale = **48 km**.
 Aucune contrainte de capacité n'est violée.

4.4. Code Python — OR-Tools (CVRP)

```

1 from ortools.constraint_solver import routing_enums_pb2
2 from ortools.constraint_solver import pywraprouting
3
4 # ---- DONNEES ----
5 # Noeud 0 = depot, Noeuds 1..6 = P1..P6
6 distance_matrix = [
7     [ 0,  3,  5,  8,  6, 10, 12], # Depot
8     [ 3,  0,  4,  7,  5,  9, 11], # P1
9     [ 5,  4,  0,  3,  6,  7,  8], # P2
10    [ 8,  7,  3,  0,  4,  5,  6], # P3
11    [ 6,  5,  6,  4,  0,  8,  9], # P4
12    [10,  9,  7,  5,  8,  0,  4], # P5
13    [12, 11,  8,  6,  9,  4,  0], # P6
14 ]
15
16 demandes      = [0, 200, 300, 250, 150, 400, 350] # 0 pour le depot
17 capacite       = 1000 # litres par camion
18 nb_camions     = 2
19 depot         = 0
20 noms          = ['Depot', 'P1(Arafat)', 'P2(Sebkha)',
21                  'P3(DarNaim)', 'P4(Teyarett)',
22                  'P5(Toujounine)', 'P6(Riyad)']
23
24 # ---- MODELE OR-TOOLS ----
25 manager = pywraprouting.RoutingIndexManager(
26     len(distance_matrix), nb_camions, depot
27 )
28 routing = pywraprouting.RoutingModel(manager)
29
30 # Callback distance
31 def dist_callback(from_idx, to_idx):

```

```

32     return distance_matrix[manager.IndexToNode(from_idx)]\
33         [manager.IndexToNode(to_idx)]
34
35 transit_idx = routing.RegisterTransitCallback(dist_callback)
36 routing.SetArcCostEvaluatorOfAllVehicles(transit_idx)
37
38 # Callback capacite
39 def demand_callback(from_idx):
40     return demandes[manager.IndexToNode(from_idx)]
41
42 demand_idx = routing.RegisterUnaryTransitCallback(demand_callback)
43 routing.AddDimensionWithVehicleCapacity(
44     demand_idx,
45     0,                # pas de capacite slack
46     [capacite] * nb_camions,
47     True,              # commencer a 0
48     'Capacite'
49 )
50
51 # Parametres de recherche
52 search_params = pywraprouting.DefaultRoutingSearchParameters()
53 search_params.first_solution_strategy = (
54     routing_enums_pb2.FirstSolutionStrategy.PATH_CHEAPEST_ARC
55 )
56 search_params.local_search_metaheuristic = (
57     routing_enums_pb2.LocalSearchMetaheuristic.GUIDED_LOCAL_SEARCH
58 )
59 search_params.time_limit.seconds = 10
60
61 # ---- RESOLUTION ----
62 solution = routing.SolveWithParameters(search_params)
63
64 # ---- AFFICHAGE ----
65 if solution:
66     print("==== Plan de distribution d'eau - Nouakchott =====")
67     dist_totale = 0
68     for vehicule in range(nb_camions):
69         index = routing.Start(vehicule)
70         tournee = []
71         charge = 0
72         dist_v = 0
73         while not routing.IsEnd(index):
74             node = manager.IndexToNode(index)
75             tournee.append(noms[node])
76             charge += demandes[node]
77             prev = index
78             index = solution.Value(routing.NextVar(index))
79             dist_v += routing.GetArcCostForVehicle(prev, index,
80                 vehicule)
81         tournee.append(noms[manager.IndexToNode(index)])
82     print(f"\nCamion {vehicule+1} : {' -> '.join(tournee)}")

```

```

82     print(f"    Charge : {charge} L / {capacite} L")
83     print(f"    Distance : {dist_v} km")
84     dist_totale += dist_v
85     print(f"\nDistance totale : {dist_totale} km")
86 else:
87     print("Aucune solution trouvee.")

```

Listing 3 – VRP avec capacité — OR-Tools Routing

Résultat attendu :

```

===== Plan de distribution d'eau - Nouakchott =====

Camion 1 : Depot -> P1(Arafat) -> P2(Sebkha) -> P4(Teyarett) ->
    Depot
    Charge : 650 L / 1000 L
    Distance : 19 km

Camion 2 : Depot -> P3(DarNaim) -> P5(Toujounine) -> P6(Riyad) ->
    Depot
    Charge : 1000 L / 1000 L
    Distance : 29 km

Distance totale : 48 km

```

4.5. Recommandations pour Nouakchott

- Planifier les tournées **tôt le matin** pour éviter la chaleur et les embouteillages.
- **Prioriser P5 et P6** (demandes élevées : 400 et 350 L) en cas de pénurie.
- Recalculer les tournées en temps réel si une demande change (OR-Tools permet cela rapidement).
- Utiliser les **données GPS réelles** de Nouakchott pour remplacer la matrice de distances estimée.

5. Conclusion générale

Ce rapport a présenté trois projets complets de Recherche Opérationnelle :

1. **La méthode du Simplexe** : fondement algorithmique de la programmation linéaire, permettant de maximiser un profit sous des contraintes de ressources.
2. **Le TSP** : problème combinatoire NP-difficile modélisant les tournées optimales, résolu efficacement avec l'heuristique d'OR-Tools.
3. **Le VRP pour la distribution d'eau à Nouakchott** : application directe et concrète à un problème mauritanien, montrant l'impact de la RO sur la vie quotidienne.

Les trois projets utilisent la bibliothèque **Google OR-Tools**, un outil professionnel utilisé par Google, DHL, et d'autres grandes entreprises pour résoudre des problèmes logistiques réels.